

2.4 Python Debugging & profiling

Edge AI Engineering Bootcamp | Analog Data

Debugging & Profiling

Learning Objectives

By the end of this session, students should be able to:

- Set breakpoints and step through code using `pdb` for interactive debugging
 - Implement production-grade logging with rotating file handlers so log files never grow unbounded on a Pi's SD card
 - Profile a Python program at the function level using `cProfile` and inspect results with `pstats`
 - Find the slowest lines inside a specific function using `line_profiler`
 - Track memory consumption per line of code using `memory_profiler`
-

Why Profiling Matters on Edge Hardware

On a desktop machine with 32GB RAM and a fast GPU, a slow or memory-hungry function is an inconvenience. On a Raspberry Pi with 1–4GB LPDDR4 and an SD card that can corrupt under power loss, it can mean:

- **Inference latency spikes** that cause missed frames in real-time video pipelines
- **Memory exhaustion** (OOM kill) crashing a model server at 3am with no operator present
- **Log files filling the SD card**, making the system unbootable

This is not academic — profiling and structured logging are production-grade survival skills, not optional polish. Every tool in this section addresses a failure mode that actually happens in deployed Edge AI systems.

1. `pdb` — Python Debugger Breakpoints

`pdb` is Python's built-in interactive debugger. It pauses execution at a chosen point and lets you inspect variables, step through code line by line, and evaluate expressions interactively.

Setting a breakpoint

```
import pdb

def preprocess(frame):
    frame = frame.astype('float32')
    pdb.set_trace()           # execution pauses here
```

```
frame = frame / 255.0
return frame
```

When execution reaches `set_trace()`, Python drops into the `pdb` prompt:

```
> preprocess.py(5)preprocess()
-> frame = frame / 255.0
(Pdb)
```

Essential `pdb` commands

Command	What it does
<code>n</code> (next)	Execute current line, stay in current function
<code>s</code> (step)	Step into the function call on this line
<code>c</code> (continue)	Resume normal execution until the next breakpoint
<code>p <expr></code>	Print the value of any expression
<code>pp <expr></code>	Pretty-print (useful for dicts, lists, NumPy arrays)
<code>l</code> (list)	Show surrounding source code
<code>q</code> (quit)	Exit the debugger and terminate the program
<code>b <line></code>	Set an additional breakpoint at a line number
<code>u</code> / <code>d</code>	Move up / down the call stack

Python 3.7+ shorthand — `breakpoint()`

```
def preprocess(frame):
    frame = frame.astype('float32')
    breakpoint()           # equivalent to pdb.set_trace() — no import needed
    frame = frame / 255.0
    return frame
```

Teaching point: Teach students to use `breakpoint()` as their first go-to when something is "returning the wrong value." The reflex of adding a breakpoint and inspecting live values is far faster and more reliable than scattering `print()` statements through the code. Also teach them to set the `PYTHONBREAKPOINT=0` environment variable in production to disable all breakpoints globally, without touching the code.

2. logging with Rotating File Handlers

Why not `print()`? `print()` only goes to stdout — it's lost the moment the terminal closes, has no timestamps, no severity levels, and no way to control verbosity per module. On a headless Pi running a `systemd` service, `print()` is effectively invisible.

Setting up structured logging

```
import logging
from logging.handlers import RotatingFileHandler

# Create a logger
logger = logging.getLogger('edge_ai_app')
logger.setLevel(logging.DEBUG)

# Console handler – for development
console_handler = logging.StreamHandler()
console_handler.setLevel(logging.INFO)

# Rotating file handler – for production
file_handler = RotatingFileHandler(
    filename='/var/log/edge_ai/inference.log',
    maxBytes=5 * 1024 * 1024,    # 5 MB per file
    backupCount=3                # keep 3 rotated files (15 MB total)
)
file_handler.setLevel(logging.DEBUG)

# Formatter – include timestamp, level, and module name
formatter = logging.Formatter(
    fmt='%(asctime)s | %(levelname)-8s | %(name)s | %(message)s',
    datefmt='%Y-%m-%d %H:%M:%S'
)
console_handler.setFormatter(formatter)
file_handler.setFormatter(formatter)

logger.addHandler(console_handler)
logger.addHandler(file_handler)
```

Using the logger in application code

```
logger.debug("Frame captured: shape=%s dtype=%s", frame.shape, frame.dtype)
logger.info("Inference complete: class=%s confidence=%.3f latency_ms=%.1f",
            class_label, confidence, latency_ms)
logger.warning("Queue near capacity: size=%d / maxsize=%d", q.qsize(), q.maxsize)
logger.error("Model inference failed: %s", str(e))
logger.critical("Memory threshold exceeded – shutting down gracefully")
```

Logging severity levels — when to use each

Level	Use case
DEBUG	Fine-grained diagnostic info useful only during development
INFO	Normal operational events (model loaded, frame processed, service started)
WARNING	Something unexpected happened but execution can continue
ERROR	A specific operation failed; the application continues
CRITICAL	Severe failure; the application may not be able to continue

Teaching point — the SD card problem: On a Pi, an unbounded log file *will* eventually fill the SD card, making the OS unbootable. `RotatingFileHandler` with `maxBytes` and `backupCount` caps total log storage. In the example above, the maximum log footprint is $5\text{MB} \times (3 \text{ backups} + 1 \text{ current}) = 20\text{MB}$ — predictable and safe. This is a mandatory pattern for any Edge AI deployment, not optional cleanup.

3. `cProfile` + `pstats` — Function-Level Profiling

`cProfile` instruments every function call in a Python program and records how many times each was called and how long it took. `pstats` provides a sorted, readable view of the results.

Running `cProfile`

From the command line (no code changes needed):

```
python3 -m cProfile -o profile_output.prof inference_app.py
```

Inside code:

```
import cProfile
import pstats

profiler = cProfile.Profile()
profiler.enable()

# --- code to profile ---
for frame in frame_stream(cap, n=100):
    result = model.infer(preprocess(frame))
# -----

profiler.disable()

stats = pstats.Stats(profiler)
stats.sort_stats('cumulative')    # sort by cumulative time spent in function + its
callees
stats.print_stats(20)             # show top 20 functions
```

Reading `pstats` output

```
ncalls  tottime  percall  cumtime  percall filename:lineno(function)
   100    0.002    0.000    4.823    0.048 inference.py:12(run_inference)
   100    0.531    0.005    0.531    0.005 preprocess.py:5(preprocess)
300000  2.841    0.000    2.841    0.000 {built-in method numpy.core...}
```

Column	Meaning
<code>ncalls</code>	Number of times this function was called
<code>tottime</code>	Time spent <i>inside</i> this function only (excluding sub-calls)
<code>cumtime</code>	Total time including all functions this one called

Column	Meaning
percall	Time per call (tottime or cumtime divided by ncalls)

Teaching point: `cumtime` is usually the most actionable column when starting — it tells you which function, including everything it triggers, is dominating total runtime. Once you find the hotspot, switch to `line_profiler` to drill down inside that specific function.

4. `line_profiler` — Line-by-Line Timing

`cProfile` tells you *which function* is slow. `line_profiler` tells you *which line inside that function* is slow.

Installation

```
pip install line_profiler --break-system-packages
```

Usage with the `@profile` decorator

```
# inference.py

@profile # line_profiler injects this decorator at runtime; no import needed
def preprocess(frame):
    frame = frame.astype('float32') # line A
    frame = frame / 255.0 # line B
    frame = cv2.resize(frame, (224, 224)) # line C
    frame = np.expand_dims(frame, axis=0) # line D
    return frame
```

Running

```
kernprof -l -v inference.py
```

Sample output

Line #	Hits	Time	Per Hit	% Time	Line Contents
5	100	12340	123.4	1.2	frame = frame.astype('float32')
6	100	18920	189.2	1.8	frame = frame / 255.0
7	100	981200	9812.0	96.1	frame = cv2.resize(frame, (224, 224))
8	100	8510	85.1	0.8	frame = np.expand_dims(frame, axis=0)

Reading the output: Line 7 (`cv2.resize`) consumes 96.1% of the function's runtime. Without `line_profiler` , a developer might waste time optimising the normalisation (line 6), which costs less than 2% — a textbook case of premature optimisation targeting the wrong line.

Teaching point: This is why profiling must come before optimisation. Intuition is wrong surprisingly often. Teach students the workflow: `cProfile` → find the slow function → `line_profiler` → find the slow line → optimise → re-profile to confirm improvement.

5. `memory_profiler` — Per-Line Memory Tracking

`memory_profiler` tracks how much RAM each line allocates or frees. On a Pi with limited RAM running a model, a memory leak or a large unnecessary intermediate array can silently grow until the OOM killer terminates the process.

Installation

```
pip install memory_profiler --break-system-packages
```

Usage

```
from memory_profiler import profile

@profile
def load_and_infer(image_path, model):
    img = cv2.imread(image_path)           # line A
    img_float = img.astype('float32')      # line B <- allocates a full copy
    img_norm = img_float / 255.0           # line C <- allocates ANOTHER copy
    batch = np.expand_dims(img_norm, axis=0) # line D
    result = model.infer(batch)            # line E
    return result
```

Running

```
python3 -m memory_profiler inference.py
```

Sample output

Line #	Mem usage	Increment	Line Contents
5	52.3 MiB	52.3 MiB	<code>img = cv2.imread(image_path)</code>
6	53.7 MiB	1.4 MiB	<code>img_float = img.astype('float32')</code>
7	55.2 MiB	1.5 MiB	<code>img_norm = img_float / 255.0</code>
8	55.2 MiB	0.0 MiB	<code>batch = np.expand_dims(img_norm, axis=0)</code>
9	127.8 MiB	72.6 MiB	<code>result = model.infer(batch)</code>

What to look for: Lines 6 and 7 each allocate a full copy of the image. For a 640×480 float32 frame that's ~3.7MB — for 1000 frames in a loop, both intermediate arrays exist simultaneously, doubling the memory footprint temporarily. The fix is in-place operations or combining steps:

```
img_norm = img.astype('float32') / 255.0 # one intermediate, not two
```

Teaching point: `memory_profiler` is especially important for catching **memory leaks in long-running inference loops**. If the `Increment` column for a line keeps growing across iterations, that line is leaking. On a Pi, a leak of even 1MB/frame will OOM-kill the process in minutes.

Profiling Tool Decision Guide

Question	Tool to reach for
"Which function is consuming the most time?"	<code>cProfile</code> + <code>pstats</code>
"Which exact line inside that function is the bottleneck?"	<code>line_profiler</code>
"Why is my Pi running out of memory?"	<code>memory_profiler</code>
"My code is returning wrong values — I need to inspect live state"	<code>pdb</code>
"I need permanent, structured, size-bounded logs in production"	<code>logging</code> + <code>RotatingFileHandler</code>

Summary Checklist for Students

- ☐ Can set a `pdb` / `breakpoint()` breakpoint and use `n`, `s`, `c`, `p`, `q` confidently
- ☐ Can configure `logging` with both a console handler and a `RotatingFileHandler`, and explain why the file handler is mandatory for Pi deployments
- ☐ Can run `cProfile` against a script and read `pstats` output to identify the top 5 slow functions
- ☐ Can decorate a function with `@profile` and run `kernprof` to get per-line timing
- ☐ Can run `memory_profiler` and identify which lines allocate the most memory
- ☐ Can explain the optimisation workflow: profile first, find the real bottleneck, optimise, re-profile